

Ejercicio 3: Arreglos dinámicos

Tabla de propiedades

#	Tipo de datos	Nombre	Visibilidad	Motivo	Clase
	int	numeroOrden	Private	Almacena número de orden	Orden
	String	nombrePropietario	Private	Almacena el nombre del propietario	Orden
	String	placaVehículo	Private	Almacena la placa del vehículo	Orden
	String	infoServicio	Private	Describe el servicio	Orden
	double	costoEstimado	Private	Almacena el costo estimado del servicio	Orden
	ArrayList<Orden>	orden	Private	Manda a llamar a orden	Controlador
	Vista	vista	Private	Manda a llamar a vista	Controlador
	Scanner	sc	Private	Permite el ingreso de datos	Vista

Tabla de métodos

#	Tipo de retorno	Nombre	Visibilidad	Parámetros	Motivo	Clase
	Void	main()	Public	String [] args	Punto de inicio	Main
		Orden	Public	int numeroOrden String nombrePropietario String placaVehículo String infoServicio double costoEstimado	constructor de Orden	Orden
	String	getInfoServicio()	Public		Obtener la descripción del servicio	Orden
	int	getNumeroOrden()	Public		Obtener el numero de orden	Orden

	String	getNombrePropietario()	Public		Obtener el nombre del propietario del vehículo	Orden
	String	getPlacaVehiculo()	Public		Obtener el número de placa del vehículo	Orden
	double	getCostoEstimado()	Public		Obtener el costo estimado del servicio	Orden
	void	setInfoServicio()	Public	String infoServicio	Dar una nueva descripción de servicio	Orden
	void	setCostoEstimado()	Public	double costoEstimado	Dar un nuevo costo	Orden
	int	mostrarMenu()	Public		Mostrar el menú	Vista
	Orden	leerOrden()	Public		Leer los datos de la orden activa	Vista
	int	leerNumeroOrden()	Public		Leer en numero de orden	Vista

	String	leerPlacaVehiculo()	Public		Leer la placa del vehículo	Vista
	String	leerNuevaInfoServicio())	Public		Leer la nueva información a dar al servicio	vista
	double	leerNuevoCostoEstimado()	Public		Leer el nuevo costo estimado	Vista
	void	mostrarOrdenes()	Public	ArrayList <Orden> ordenes	mostrar las ordenes registradas	Vista
	void	mostrarOrden()	Public	Orden orden	mostrar una orden por su numero	Vista
	void	mostrarOrdenPlaca()	Public	ArrayList <Orden> ordenes	mostrar las órdenes de una placa específica	Vista
	void	mostrarValorOrdenesActivas()	Public	double total	mostrar el valor total de órdenes activas	Vista
	void	mostrarPromedio()	Public	double promedio	mostrar el costo promedio de las ordenes activas	Vista

	void	mostarCostoMayor()	Public	Orden orden	mostrar la orden con mayor costo estimado	vista
	void	mostarCantidadOrdenes()	Public	int cantidadOrdenes	mostrar la cantidad de ordenes registradas	vista
	void	mensaje()	Public	String texto	mostrar un mensaje determinando el resultado de la operación realizada	Vista
	void	error()	Public	String texto	Mostrar un mensaje cuando hay un error	Vista
	void	iniciar()	Public		Iniciar el controlador	Controlador
	void	registrarOrden	Public		registrar una nueva orden	Controlador
	void	consultaOrdenes()	Public		consultar las ordenes registradas	Controlador

	void	busquedaOrden()	Public		Buscar una orden por su numero	Controlador
	void	modificarOrden()	Public		Modificar una de las órdenes registradas	Controlador
	void	cancelarOrden()	Public		Cancelar una orden	Controlador
	void	consultaOrdenPlaca()	Public		Consultar las órdenes de una placa específica	Controlador
	void	consultaReporteCostos()	Public		consultar el valor total de órdenes activas y el costo promedio	Controlador
	void	consultarCostoMayor()	Public		consultar la información de la orden con costo estimado más alto	controlador
	void	cantidadOrdenes()	Public		consultar número de órdenes registradas	Controlador

	void	salir()	Public		terminar el programa	Controlador
	Orden	buscarOrdenNumero()	Private	int numeroOrden	realizar la búsqueda de la orden por su numero	Controlador
	ArrayList<Orden>	buscarOrdenPlaca()	Private	String placaVehiculo	Buscar las ordenes correspondientes a una misma placa	Controlador
	double	calculoTotalOrdenes()	Private		Calcular el valor del costo estimado de todas las ordenes registradas	Controlador
	double	calculoPromedioOrdenes()	Private		Calcular el promedio del costo estimado de las ordenes registradas	Controlador
	Orden	determinarMayorCosto()	Private		Recorrer las ordenes y determinar cual es el que tiene mayor costo	Controlador

8. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores iniciales les asignará?

Los valores se proveerán al consultar al usuario los datos que desea agregar por medio del Scanner.

9. ¿Cómo identificará una orden específica dentro de la colección?

Una orden específica será identificada por medio de índices dentro del ArrayList

10. ¿Cómo agregará y eliminará órdenes de la colección dinámica?

Por medio de los métodos ya establecidos de la colección dinámica de add y remove

11. ¿Cómo realizará la búsqueda de todas las órdenes asociadas a una misma placa?

Al recorrer el array y encontrando la similitud en placas de las ordenes

12. ¿Cómo recorrerá la colección para calcular el valor total y el costo promedio de las órdenes?

Se recorrerán los costos estimados de las ordenes por medio de for each para realizar la sumatoria.

En el caso del promedio se obtendrá el valor obtenido en el total y se dividirá entre la cantidad de ordenes registradas

13. ¿Cómo determinará cuál es la orden con el costo estimado más alto?

Por medio de un ciclo en el que se evalúa cada costo estimado de las ordenes, y estableciendo si el primero es mayor que el segundo y así consecutivamente hasta recorrer toda la lista

14. ¿Qué situaciones dentro del programa pueden producir una excepción?

Argumentos no válidos (IllegalArgumentException), recorrer fuera del índice (IndexOutOfBoundsException), elegir una opción en null (NullPointerException), problemas de aritmética tal como la división entre 0.

En este caso podrían producirse principalmente argumentos no válidos.

15. ¿Qué operaciones serán protegidas utilizando try-catch?

Evaluar que los valores ingresados no sean nulos, 0 o que ya estén registrados

16. ¿Qué acción realizará mediante el bloque finally y por qué debe ejecutarse independientemente del resultado de la operación?

Se realizará el display de un mensaje que indique que la operación ha finalizado, se ejecuta independientemente del resultado porque se debe ejecutar el código a pesar de una excepción